

Theoretical Foundations of the WAFT Architecture

A Synthesis of Recursive Cognition, Evolutionary
Dynamics, and Formal Epistemology

Integrating Six Foundational Disciplines

- Reflexion Architectures
- Ludology (D&D 5e)
- Literate Programming
- Computational Phylogenetics
- Epistemic Logic
- Typst Automation

January 24, 2026

Executive Summary

The emergence of autonomous agentic architectures demands a rigorous theoretical framework that transcends the ad-hoc prompt engineering currently dominating the field. The WAFT project represents a convergence of six distinct yet interlinked disciplines to solve the problems of agent self-improvement and lineage tracking.

Core Synthesis

This report provides an exhaustive technical analysis of:

- **Reflexion architectures** for recursive verbal reinforcement
- **Computational Phylogenetics** for ancestral state reconstruction
- **Ludology** (D&D 5e) for probabilistic benchmarking
- **Epistemic Logic** for formal reasoning about uncertainty
- **Literate Programming** incorporating Quines for self-documentation
- **Typst automation** as the rendering layer

The analysis posits that by synthesizing recursive verbal reinforcement (Reflexion) with ancestral state reconstruction (Phylogenetics), one can engineer a system that not only improves through trial and error but maintains a mathematically rigorous lineage of its own cognitive evolution.

Discipline	Contribution	Status
Reflexion	Verbal reinforcement loop	✓
Phylogenetics	Evolutionary lineage tracking	✓
Ludology	Quantified difficulty scaling	✓
Epistemic Logic	Formal knowledge representation	✓
Literate Programming	Self-documenting code	✓
Typst	Dynamic report generation	✓

Table 1: Six Pillars of the WAFT Architecture

Furthermore, the integration of formal game mechanics provides a probabilistic environment for valid benchmarking, while Epistemic Logic offers the syntax for the agent's internal reasoning about its own uncertainty. The principles of Literate

Programming and Quines, implemented via Typst, ensure that the system remains self-documenting and autopoietic—capable of reproducing its own operational context alongside its outputs.

Reflexion: The Architecture of Verbal Reinforcement

The “Reflexion” architecture represents a fundamental paradigm shift in the training and operation of Large Language Model (LLM) agents. While traditional Reinforcement Learning (RL) has focused on optimizing synaptic weights through scalar reward signals—a process that is computationally expensive and interpretability-poor—Reflexion introduces the concept of **verbal reinforcement**.

Key Innovation

Reflexion converts the “black box” failure of a neural network into a **semantic object** that the network itself can process and learn from.

This mechanism utilizes linguistic feedback to induce self-correction, effectively converting errors into learning signals. This architecture is the primary cognitive engine driving the “Evolutionary Cycle” of the WAFT system, providing the mechanism for mutation and selection within the agent’s cognitive runtime.

The Limitations of Traditional Reinforcement Learning

To understand the necessity of Reflexion within WAFT, one must first analyze the deficiencies of standard RL approaches when applied to large language models.

In traditional RL, an agent interacting with an environment receives a reward signal r_t and updates its policy parameters θ to maximize the expected cumulative reward:

$$E \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (1)$$

However, in the context of LLMs with billions of parameters, this approach faces three critical bottlenecks:

Bottleneck	Description
Computational Intractability	Fine-tuning a 70B+ parameter model for every specific task instance is computationally prohibitive. The gradient updates required are massive, and the memory overhead for maintaining optimizer states is significant.
Episodic Amnesia	Weight updates are slow to converge. An agent might require thousands of episodes to “learn” a simple heuristic, and this learning is implicit—buried in the floating-point weights—rather than explicit.
Opacity	When an RL agent improves, it is often impossible to determine why. Did it learn the rule, or did it overfit to a statistical artifact? This lack of interpretability is fatal for safety-critical systems.

Table 2: Three Critical Bottlenecks in Traditional RL for LLMs

Reflexion addresses these issues by **decoupling “learning” from “weight updates”**. Instead of updating θ , Reflexion updates the context c provided to the fixed policy $\pi_\theta(\cdot \mid c)$. The agent “learns” by accumulating a verbal memory of its own experiences.

The Operational Loop of Reflexion

The Reflexion framework operates on a cyclical process distinct from standard Chain-of-Thought (CoT) prompting. While CoT encourages intermediate reasoning steps, it is typically an open-loop system; if the reasoning is flawed, the model hallucinates a conclusion without correction.

Reflexion formalizes a **closed feedback loop** through three distinct components:

The Actor and the Trajectory

The Actor is the LLM agent instantiated to solve a specific task within the WAFT “Scint Gym.” In a standard trajectory, the Actor generates an action a_t based on the current state s_t .

In the Reflexion paradigm, the trajectory τ includes not just state-action pairs, but also **linguistic feedback** and **memory traces**.

The prompt structure for the Actor at trial t becomes:

$$\text{Prompt}_t = \text{Task Description} + \text{Context} + \text{Memory}(\tau_{0:t-1}) \quad (2)$$

This structure allows the agent to “remember” that it previously tried Strategy A and failed, thus incentivizing it to try Strategy B, mimicking the cognitive flexibility of biological problem-solving.

The Evaluator: Ground Truth and Hallucination Checks

The Evaluator serves as the critic in the loop, providing a fidelity check on the Actor’s output.

Evaluator Types in WAFT

Deterministic Evaluation: For coding tasks (e.g., LeetCodeHardGym), the Evaluator is a compiler or unit test suite. Returns binary success/failure plus error message.

Heuristic Evaluation: For reasoning tasks (e.g., HotPotQA), a separate LLM checks for exact match answers or logical consistency.

Epistemic Evaluation: Assesses confidence and safety using Epistemic Logic vectors.

The output of the Evaluator is a scalar reward r , but its primary function is to trigger the reflection process if r indicates failure. The semantic content of the evaluation is passed to the Self-Reflection model.

The Self-Reflection Mechanism

This is the core innovation relevant to WAFIT’s evolutionary cycle. Upon receiving a failure signal, the Self-Reflection model analyzes the trajectory τ and the error signal, generating a summary h —a “reflection.”

This reflection is not merely a restatement of the error. It is a **synthesis of intent versus outcome**. For example:

“I failed to increment the counter variable i inside the while loop, causing an infinite loop.”

This reflection h is stored in a sliding window memory buffer. For the next trial, the Actor is prompted with the accumulated reflections:

$$\text{Context}_{t+1} = [h_1, h_2, \dots, h_t] \quad (3)$$

This creates a **gradient of semantic improvement**. The agent distinguishes between a “mistake” (an execution error) and a “deep error” (a fundamental misunderstanding), effectively “gradienting” its own semantic space without modifying its weights.

Comparative Analysis: Reflexion vs. Traditional RL

The distinction between Reflexion and traditional RL is not merely implementation but architectural philosophy.

Feature	Traditional RL	Reflexion
Optimization Target	Neural Weights (θ)	Context Buffer (c)
Feedback Signal	Scalar Reward ($r \in \mathbb{R}$)	Semantic Text + Scalar
Memory Type	Implicit (Weights)	Explicit (Text Buffer)
Sample Efficiency	Low (Thousands)	High (Single-digit)
Interpretability	Black Box	Human-Readable Logs
Transferability	Requires Re-training	Text Buffer Copy

Table 3: Architectural Comparison: Traditional RL vs. Reflexion

This comparison highlights why Reflexion is superior for the WAFT “Scint Gym”:

- The **sample efficiency** allows the agent to evolve rapidly within a single session
- The **transferability** means that a successful “Reflection” can be extracted and injected into other agents, simulating **horizontal gene transfer**

Theoretical Limits and Second-Order Insights

Research indicates that Reflexion is not a panacea. The capacity for self-correction is bounded by the model’s intrinsic capability to recognize an error when pointed out.

Hallucination Loops

If the model is fundamentally incapable of reasoning about the domain (e.g., lack of epistemic access to necessary facts), “hallucination loops” can occur where the agent reflects but invents incorrect reasons for failure.

Furthermore, the **context window** limits the number of reflections an agent can hold. This necessitates a “pruning” or “summarization” mechanism—a form of **cognitive garbage collection** where outdated reflections are discarded, akin to forgetting in biological systems.

This leads to the requirement for a phylogenetic tracking system to manage long-term history, which is addressed by Computational Phylogenetics.

Computational Phylogenetics: Mapping the Evolutionary Cycle

If Reflexion provides the mechanism for an agent to “mutate” its behavior in response to error, Computational Phylogenetics provides the mathematical framework to map, track, and reconstruct these changes over time.

In the context of WAFT, the “Evolutionary Cycle” is not merely a metaphor but a **directed graph of agent states**, where each “generation” is a refined prompt-state derived from an ancestor.

Ancestral State Reconstruction (ASR)

Ancestral reconstruction is the extrapolation back in time from measured characteristics of individuals (tips of the tree) to their common ancestors (internal nodes).

Genomic Analogy

In biology, ASR reconstructs DNA sequences.

In WAFT, ASR reconstructs the “**cognitive state**” or “**source code**” of an agent that led to a successful divergence.

The WAFT system treats the “prompt” or “source code” of an agent as a **genome**. As the agent undergoes Reflexion, it accumulates changes (mutations). When a specific configuration solves a difficult problem in the Scint Gym, it represents a “fitness peak.”

ASR allows the system to determine which specific mutations contributed to that peak and which were merely “genetic hitchhikers” (neutral mutations).

Maximum Parsimony (Fitch’s Algorithm)

Maximum Parsimony (MP) operates on Occam’s Razor: the best evolutionary tree is the one that requires the **fewest evolutionary changes**.

This is particularly relevant for WAFT because prompt engineering benefits from conciseness; we assume that the optimal prompt is the simplest one that yields the desired behavior.

Fitch’s algorithm operates in two passes:

1. **Postorder Traversal (Tips to Root):** For each internal node n , the set of possible states S_n is determined by its children l and r :

$$S_n = \begin{cases} S_l \cap S_r & \text{if } S_l \cap S_r \neq \emptyset \\ S_l \cup S_r & \text{if } S_l \cap S_r = \emptyset \end{cases} \quad (4)$$

If the intersection is empty, the union is taken, implying a mutation occurred at this node.

2. **Preorder Traversal (Root to Tips):** This pass assigns specific states to the ambiguity identified in the first pass.

Application to WAFT

When an agent forks into multiple parallel attempts (beam search), Fitch’s algorithm can determine the “**Minimal Viable Prompt**” that serves as the common ancestor for all successful branches.

This allows the system to prune redundant prompt engineering instructions.

Maximum Likelihood and Bayesian Inference

While Parsimony is intuitive, it often fails when rates of evolution vary (long branch attraction). Maximum Likelihood methods use explicit models of evolution (Markov Chains) to calculate the probability of the data given a tree.

The **Mk model** (Markov k-state) is particularly relevant for discrete traits. It models transitions between k states using a transition rate matrix Q :

$$P(t) = e^{Qt} \quad (5)$$

where $P(t)$ is the probability matrix of transitioning between states over time t .

In the WAFT Evolutionary Cycle, “time” (t) is represented by the **number of Reflexion steps** or “generations.”

A Bayesian approach allows the system to account for uncertainty in the tree topology itself—acknowledging that we may not know exactly which prompt caused which improvement if multiple agents are collaborating asynchronously.

Tree Search Spaces and Topology

Finding the optimal tree is an NP-hard problem. The search space for n taxa (distinct agent versions) is $(2n - 3)!!$ (double factorial).

To navigate this, algorithms use heuristic rearrangements:

Algorithm	Description
NNI	Nearest Neighbor Interchange: Swapping subtrees across an internal branch. Used for local optimization of prompt clusters.
SPR	Subtree Prune and Regraft: Cutting a branch and reattaching it elsewhere. Models “transplanting” a skill from one agent to another.
TBR	Tree Bisection and Reconnection: Breaking the tree into two and reconnecting them.

Table 4: Tree Rearrangement Algorithms

WAFT Insight

The “Scint Gym” can be visualized as a **phylogeny search space**. Each exercise creates a new branch.

High-performing agents occupy local optima in the tree space. The system uses SPR-like moves to “transplant” a successful strategy from one problem

domain to another (**lateral gene transfer**), accelerating evolution beyond simple vertical inheritance.

The Coalescent and Genetic Drift

Population genetics introduces the concept of the **Coalescent**—tracing lineages backward to a Most Recent Common Ancestor (MRCA).

In a population of agents, “Genetic Drift” occurs when certain successful prompts become fixed in the population purely by chance, not necessarily because they are optimal (stochastic sampling).

Prompt Drift

WAFT must guard against “**Prompt Drift**,” where the agent’s instructions drift away from original safety guidelines due to successive iterations of self-optimization.

The “Sentinel Safety Gates” serve as selection pressure to counteract this drift, pruning lineages that violate epistemic or safety constraints.

By monitoring the “Effective Population Size” (N_e) of the prompt variations, WAFT can ensure that the gene pool of ideas remains diverse enough to handle novel tasks.

Ludology and the “Scint Gym”: Quantifying Uncertainty

The “Scint Gym” requires a robust mechanism for simulating difficulty and measuring success probabilities. The System Reference Document (SRD) of Dungeons & Dragons 5th Edition (D&D 5e) provides a surprisingly sophisticated, yet computationally inexpensive, framework for modeling bounded accuracy and probabilistic outcomes.

Not Gamification for Engagement

This is **not** gamification for engagement; it is the adoption of a rigorous simulation engine for agent benchmarking.

The Mathematics of the d20 System

The core mechanic of D&D 5e is the d20 roll:

$$\text{Result} = \text{d20} + \text{Modifiers} \geq \text{DC} \quad (6)$$

This linear probability distribution (each face has a 5% probability) contrasts with the bell curves of dice pool systems (e.g., 3d6). However, the interactions of this linear system with “Bounded Accuracy” create a stable simulation environment.

Difficulty Class (DC) Scaling

The 5e SRD defines discrete tiers of difficulty. In the Scint Gym, tasks are assigned a DC based on their complexity.

Difficulty	DC	Success (Mod +0)	Success (Mod +5)
Very Easy	5	80%	100%
Easy	10	55%	80%
Medium	15	30%	55%
Hard	20	5%	30%
Very Hard	25	0%	5%
Impossible	30	0%	0%

Table 5: DC Scaling and Success Probabilities

In the Scint Gym, the “Skill” of an agent is its **Modifier**. An agent with a +5 modifier (high proficiency) interacting with a DC 15 task has:

$$P(\text{Success}) = \frac{21 - (15 - 5)}{20} = \frac{11}{20} = 55\% \quad (7)$$

This creates a “**Bounded Accuracy**” system where numbers do not inflate infinitely. This keeps the Scint Gym calibration stable across different generations of agents.

Advantage and Disadvantage: Non-Linear Probability

The most significant statistical innovation in 5e is the Advantage/Disadvantage mechanic, which replaces granular modifiers with a boolean state change:

- **Advantage:** Roll 2d20, take the highest ($\max(d_1, d_2)$)
- **Disadvantage:** Roll 2d20, take the lowest ($\min(d_1, d_2)$)

The probability mass function (PMF) shifts dramatically. For a Target Number (T) on the die:

$$P(X \geq T \mid \text{Advantage}) = 1 - (1 - P(X \geq T))^2 \quad (8)$$

$$P(X \geq T \mid \text{Disadvantage}) = P(X \geq T)^2 \quad (9)$$

Key Insight for Scint Gym

At DC 10 (requiring roll of 10+, 55% base chance):

- **Advantage** boosts success to $\approx 79.8\%$ (equivalent to +5 modifier)
- **Disadvantage** drops success to $\approx 30.2\%$ (equivalent to -5 modifier)

However, at extremes (DC 20), Advantage acts like only a +1 or +2 modifier. This nonlinearity models “**consistency**” rather than raw power.

In WAFT, **Reflexion** can be modeled as granting “**Advantage**” on the next attempt. The agent uses its memory to simulate a second roll, picking the better outcome.

Conversely, “**Hallucination**” acts as **Disadvantage**.

The Scint Gym can explicitly calculate the “Epistemic Modifier” of an agent by observing its success rate against known DCs and solving for the implied modifier.

Skill Checks as Agent Benchmarks

The SRD defines ability checks (Strength, Intelligence, Wisdom). In WAFT, these map to agent capabilities:

D&D Ability	WAFT Agent Capability
Intelligence (INT)	Reasoning, Logic, Code Generation
Wisdom (WIS)	Perception, Safety Awareness, Context Retention
Charisma (CHA)	User Persuasion, Formatting, Tone

Table 6: D&D Ability to Agent Capability Mapping

By assigning DCs to these specific vectors, the Scint Gym can generate a “**Character Sheet**” for the AI Agent:

- A “Level 1” agent might have +2 INT
- A “Level 20” agent (after many Reflexion cycles) has +11

This allows for **Monte Carlo simulations** of agent performance. Instead of running a costly agent 100 times, if we know its “Modifier” and the Task “DC,” we can analytically predict its pass rate.

The Scint Gym essentially runs the agent through a “dungeon” of tasks (e.g., “The Caverns of Recursion” or “The Tower of Safety”), where each room is a prompt with a specific DC. The agent’s survival through the dungeon determines its fitness for the Evolutionary Cycle.

Epistemic Logic: Formalizing the Known and the Unknown

While Reflexion handles the improvement of knowledge, and Phylogenetics tracks the history of knowledge, Epistemic Logic provides the formal syntax to represent what the agent **knows**, **believes**, and **considers possible**.

This is the operating system of the “Epistemic Ledger.”

Modal Logic and the Axioms of Knowledge

Epistemic logic extends propositional logic with modal operators K_a (Agent a knows) and B_a (Agent a believes).

To build a safe agent, WAF T must rigorously define these operators using axiomatic systems.

The standard system for **Knowledge** is **S5**, defined by the axioms:

Axiom	Name	Description
K	Distribution	$K(\varphi \rightarrow \psi) \rightarrow (K\varphi \rightarrow K\psi)$ If I know P implies Q, and I know P, I know Q.
T	Truth	$K\varphi \rightarrow \varphi$ If I know P, P must be true. (Veridicality)
4	Positive Introspection	$K\varphi \rightarrow KK\varphi$ If I know P, I know that I know P.
5	Negative Introspection	$\neg K\varphi \rightarrow K\neg K\varphi$ If I don't know P, I know that I don't know P.

Table 7: S5 Modal Logic Axioms for Knowledge

The standard system for **Belief** is **KD45**:

- **D (Consistency):** $B\varphi \rightarrow \neg B\neg\varphi$ — I cannot believe P and not-P simultaneously
- **4 & 5:** Same as above, but for belief

Key Difference

Axiom T is dropped for belief because **beliefs can be false**.

The “Logical Omniscience” Problem and WAFT

A major issue with S5/KD45 in AI is “**Logical Omniscience**” (Axiom K). Real agents are computationally bounded; they may know the rules of chess but not “know” the optimal move, even though it is a logical consequence of the rules.

WAFT's Solution

WAFT must implement a **Resource-Bounded Epistemic Logic**.

The Empirica tool’s “13 epistemic vectors” likely represent a **vector space model** of knowledge that replaces binary modal operators with continuous confidence intervals.

Instead of $K\varphi$, the system tracks:

$$P(\varphi \mid \text{Context}) > \text{Threshold} \quad (10)$$

The “Reflexion” step is essentially an operation to satisfy **Axiom 5** (Negative Introspection). The agent realizes $\neg K(\text{Solution})$, and this realization $K\neg K$ triggers the query for external information.

Without this logic, the agent would proceed in ignorance.

Dynamic Epistemic Logic (DEL) and Multi-Agent Systems

In a multi-agent WAFT implementation, DEL becomes critical. DEL models how knowledge changes upon “Public Announcements” or “Private Communications.”

Public Announcement Logic (PAL): $[\!|\varphi|\!]\psi$ — “After φ is truthfully announced, ψ becomes true.”

When the Evaluator returns an error message, this is a **Public Announcement**. The agent updates its Kripke Model (the set of possible worlds) by eliminating all worlds where the solution was correct.

This **pruning of the epistemic state space** is what constitutes “learning” in the logical sense.

Group Knowledge

- $E_G\varphi$: Everyone in group G knows φ
- $C_G\varphi$: It is **Common Knowledge** that φ (Everyone knows that everyone knows...)

Achieving Common Knowledge is the goal of the “Project Memory” and “Dynamic Context Loader.” Without Common Knowledge, agents in the Scint Gym cannot effectively collaborate on complex tasks, as they will lack a shared reference frame.

The 13 Epistemic Vectors

The WAFT system extends classical logic into 13 specific dimensions. These vectors likely include:

Core Vectors:	Extended Vectors:
1. Factual Accuracy (Truth value)	7. Safety (Sentinel compliance)
2. Confidence (Probabilistic belief)	8. Temporal Validity
3. Justification (Traceability)	9. Source Provenance
4. Completeness (Domain coverage)	10. Consensus Agreement
5. Coherence (Logical consistency)	11. Computational Cost
6. Relevance (Engagement)	12. Uncertainty Quantification
	13. Meta-Cognitive Awareness

Figure 1: The 13 Epistemic Vectors (Inferred)

These vectors form the **Epistemic State** of the agent, which is the object being evolved in the Evolutionary Cycle. The Scint Gym measures the agent's performance across these vectors, providing a **multidimensional fitness landscape**.

Literate Programming and Quines: The Autopoietic Structure

The structural paradigm of WAFT is “**Literate Programming**,” a concept introduced by Donald Knuth. This methodology inverts the traditional code-comment relationship: instead of writing code and adding comments, one writes a narrative document that contains code snippets.

This ensures that the agent’s **rationale is always preserved alongside its function**.

Knuth’s WEB vs. Modern Tools

Knuth’s original WEB system used:

- **Tangle:** To extract compilable code (rearranges code blocks for machine order)
- **Weave:** To generate TeX documentation (formats for human reading in psychological order)

WAFT utilizes this to maintain “**Living Documentation**.” In the WAFT Evolutionary Cycle, the “genome” of the agent is not just the Python script; it is the **Literate Source**—the markdown/Typst document that explains **why** the agent works the way it does.

Modern Integration

Modern tools like `nbdev` or `mkdocstrings` approximate this, but WAFT integrates it at the **architectural level**.

The Quine: Self-Replication and Autopoiesis

A **Quine** is a program that, when executed, outputs its own source code:

$$P(\emptyset) = P \quad (11)$$

This is a realization of **Kleene’s Recursion Theorem**. In the context of WAFIT, the “Evolutionary Cycle” implies a “**Constructive Quine**.” The agent does not just output its source; it outputs a **modified version** of its source (a mutation).

Introspection Mechanisms

To achieve this, the system leverages Python’s `inspect` module:

```
import inspect

# Allows the agent to read its own code as a string
source = inspect.getsource(object)

# Allows the agent to read its own documentation
docs = inspect.getdoc(object)
```


The Reflexive Quine Loop

The operational cycle of a WAFT agent is an **Ouroboros**:

The Five Phases of the Quine Loop:

- | | |
|-------------------------|---|
| 1. Introspection | The agent reads its own source code using <code>inspect</code> |
| 2. Reflexion | The agent performs a task in the Scint Gym and analyzes performance |
| 3. Mutation | The agent modifies the source code string to incorporate new learning |
| 4. Reproduction | The agent outputs the new Literate Document (Generation N+1) |
| 5. Birth | Typst compiles the document into the new Agent executable |

Listing 1: The Reflexive Quine Loop

This structure ensures that **documentation never drifts from the code**, because the documentation is the generative source of the code. It creates an **Autopoietic System**—a system capable of reproducing and maintaining itself.

Typst: The Rendering and Automation Layer

The final component is **Typst**, a modern programmable typesetting system that replaces LaTeX in the WAFT pipeline. Typst is not just a markup language; it is a **scripting language** with variables, loops, and functions, making it the ideal “Weaver” for the Literate Programming/Quine cycle.

Scripting and Automation

Unlike LaTeX, which utilizes macro expansion (often brittle), Typst uses a **functional scripting paradigm**.

```
#let evolutionary_generation = 42
#let success_rate = 0.91

#show heading: it => [
  Generation #evolutionary_generation: #it
]
```

This capability allows the WAFT system to **dynamically generate reports** where the data (from Scint Gym) directly alters the formatting and structure of the document.

Dynamic Reports

The report is not a static artifact; it is a **dynamic view** of the agent’s internal state.

Visualization: Fletcher and Genotypst

Visualizing the “Evolutionary Cycle” requires advanced graphing capabilities embedded directly in the document generation process.

Fletcher: Logic Visualization

Fletcher is a Typst package for drawing diagrams with nodes and edges. It uses a coordinate system to draw state machines and flowcharts.

In WAFT, Fletcher is used to render the **Reflexion Trace**—visualizing the loop of Action $\rightarrow$ Observation $\rightarrow$ Reflection.

```
#import "@preview/fletcher:0.5.0" as fletcher: diagram, node, edge

#diagram(
  node((0,0), [Action]),
  edge("->"),
  node((1,0), [Observation]),
  edge("->"),
  node((2,0), [Reflection]),
  edge("->", bend: 130deg),
)
```

This allows the agent to visually debug its own logic flow in the generated report.

Genotypst: Phylogenetic Rendering

Genotypst is a bioinformatics package for Typst. It parses FASTA/Newick files and renders Phylogenetic Trees.

Application: WAFT uses Genotypst to render the actual lineage of the agent. As the agent forks and evolves, the system saves the lineage in Newick format:

```
((Agent_V1:0.1, Agent_V2:0.2):0.5, Agent_V3:0.8);
```

Genotypst then automatically renders this tree in the final report. This visual feedback allows human overseers to see the “**family tree**” of the agent swarm and identify which branches are flourishing in the Scint Gym.

The Typst “Quine”

Because Typst is scriptable, a Typst document can **read its own source file** and render it. This closes the loop on the Literate Programming requirement.

The “WAFT Report” is a Typst document that contains:

- The Python code for the agent
- The D&D math for the gym
- The Genotypst code for the tree

All compiled into a **single PDF** that is the agent’s body of knowledge.

Synthesis: The WAFT System Architecture

Integrating these six fields reveals the complete “Evolutionary Cycle” and “Scint Gym” of WAFT. The system is not merely a collection of tools but a **unified architecture for artificial cognition**.

The Scint Gym (Ludology + Epistemic Logic)

The Scint Gym is a training environment where tasks are rated by D&D 5e Difficulty Classes (DC).

Scint Gym Operational Flow:

1. **Input:** An Agent (with “Ability Scores” derived from previous performance)
2. **Task:** A coding or reasoning problem with an assigned DC (e.g., Leetcode Hard = DC 25)
3. **Mechanism:**
 - The agent attempts the task
 - **Epistemic Check:** Agent calculates confidence ($B_a\varphi$)
 - If Confidence < Threshold, trigger Reflexion
 - **Reflexion (Advantage):** Generate self-reflection, effectively rolling 2d20
 - Take the best reasoning path
4. **Outcome:** Success or Failure is recorded
5. **Bayesian Update:** Agent’s internal “Ability Score” is updated based on DC and outcome

Figure 2: Scint Gym Operational Flow

The Evolutionary Cycle

This cycle describes how the agent evolves over generations:

Generation N (Parent): A Literate Program (Quine) containing the agent's source and current Epistemic State

Mutation (Reflexion): Agent encounters high-DC problem in Scint Gym. Reflects, learns new strategy

Encoding: Verbal reflection is “compiled” into code change or permanent prompt instruction

Replication (Quine): Agent writes new Literate Document (Generation N+1) containing new instruction

Phylogeny: Parent-child relationship recorded in Newick tree

Selection: If new agent performs better (higher $P(\text{success})$ against DC), it is kept. Otherwise pruned.

Rendering: Typst compiles new document, visualizes updated Phylogenetic tree and Logic flow

Figure 3: The Seven Phases of the Evolutionary Cycle

Conclusion

The WAFT architecture represents a sophisticated attempt to create an **Autopoietic Cognitive System**. By moving beyond simple gradient descent (Reflexion) and treating the agent’s development as a measurable evolutionary process (Phylogenetics) within a standardized probabilistic environment (Ludology), WAFT achieves a high degree of rigorous control.

Architectural Achievement

Key Innovations:

1. **Reflexion** ensures continuous improvement without weight updates
2. **Phylogenetics** tracks lineage and identifies successful mutations
3. **Ludology** provides stable, quantified benchmarking
4. **Epistemic Logic** ensures sound reasoning about uncertainty
5. **Literate Programming** maintains transparency and traceability
6. **Typst** enables dynamic self-documentation

The “Scint Gym” is not just a test; it is the **selection pressure** that drives the “Evolutionary Cycle” of the Quine-based agent towards higher cognitive capability.

The convergence of these six disciplines creates a system that is simultaneously:

**Self-Improving • Traceable • Quantifiable • Logical
• Transparent • Autopoietic**

Mathematical Appendix

D&D 5e Probability Formulas

To rigorously simulate the “Scint Gym,” WAF T employs the exact probability mass functions derived from the d20 system.

For a given Modifier M and Difficulty Class DC:

Normal Roll:

$$P_{\text{normal}} = \frac{21 - (\text{DC} - M)}{20} \quad (12)$$

Constraint: $0.05 \leq P \leq 0.95$ (Natural 1 is fail, Natural 20 is success)

Advantage (Reflexion):

$$P_{\text{adv}} = 1 - (1 - P_{\text{normal}})^2 \quad (13)$$

Substituting the normal probability:

$$P_{\text{adv}} = 1 - \left(1 - \frac{21 - \text{DC} + M}{20}\right)^2 \quad (14)$$

$$P_{\text{adv}} = 1 - \left(\frac{\text{DC} - M - 1}{20}\right)^2 \quad (15)$$

Disadvantage (Hallucination/Noise):

$$P_{\text{dis}} = P_{\text{normal}}^2 \quad (16)$$

$$P_{\text{dis}} = \left(\frac{21 - \text{DC} + M}{20}\right)^2 \quad (17)$$

These formulas allow the Scint Gym to calculate the **expected value** of an agent’s performance analytically, validating the Monte Carlo results obtained from actual trials.

Bayesian Phylogenetic Update

The probability of a specific evolutionary tree T (lineage of prompts) given the performance data D (success/fail in Gym) is calculated using Bayes' Theorem:

$$P(T|D) = \frac{P(D|T) \cdot P(T)}{P(D)} \quad (18)$$

Where:

- $P(D|T)$ is the **Likelihood**. Calculated using the Mk Model on the tokens of the prompts. If prompt A differs from prompt B by 3 tokens, the likelihood of this transition is derived from the transition rate matrix Q .
- $P(T)$ is the **Prior**. WAFIT uses a prior that favors simpler trees (parsimony) or trees that align with known “safe” prompt structures.
- $P(D)$ is the **marginal likelihood** (sum over all possible trees), which serves as a normalizing constant.

Epistemic Logic Axiom S5

The semantic model for the agent's knowledge is based on Kripke frames $\mathcal{F} = \langle W, R \rangle$.

The S5 system, which governs “Knowledge,” requires the accessibility relation R to be an **equivalence relation**:

1. **Reflexive:** $\forall w \in W : wRw$ (Supports Axiom T: If known, then true in current world)
2. **Symmetric:** $\forall w, v \in W : wRv \Rightarrow vRw$ (Supports Axiom B)
3. **Transitive:** $\forall w, v, u \in W : (wRv \wedge vRu) \Rightarrow wRu$ (Supports Axiom 4: Positive Introspection)

WAFIT's “Epistemic Vectors” essentially define the **topology** of this R relation:

- A “Confident” agent has a very restrictive R (few accessible worlds)
- An “Uncertain” agent has a broad R (many possible worlds)

Detailed Analysis of Component Interactions

To fully appreciate the robustness of the WAFT architecture, one must analyze the pairwise and higher-order interactions between these six components. These intersections are where the **emergent properties** of the system arise.

Interaction: Reflexion $\leftrightarrow$ Epistemic Logic

The relationship between Reflexion and Epistemic Logic is causal and foundational. Reflexion is the **procedural implementation** of the epistemic axiom of Negative Introspection (Axiom 5).

Theoretical Conflict

Axiom 5: $\neg K\varphi \rightarrow K\neg K\varphi$ implies that an agent always knows when it is ignorant.

In current LLM architectures, this is demonstrably false; models frequently suffer from “hallucinations,” confidently stating false propositions. This violation of Axiom 5 is a primary safety risk.

WAFT Solution: The Reflexion loop forces the system to mechanically simulate Axiom 5:

- The **Evaluator** (external compiler or critic) acts as an oracle providing the ground truth $\neg\varphi$
- The **Self-Reflection** step forces the agent to accept $K\neg\varphi$ (I know it is false) and effectively $K\neg K_{\text{prev}}$ (I know I didn’t know)

Epistemic Vectors: The “13 vectors” likely act as the state space for the Reflexion policy. Instead of reflecting on everything (inefficient), the agent only triggers the expensive Reflexion process on vectors where:

$$\text{Confidence} < \text{Safety Threshold} \quad (19)$$

This optimization prevents infinite regression loops and focuses computational resources on areas of epistemic uncertainty.

Interaction: Ludology ↔ Computational Phylogenetics

The Scint Gym (Ludology) provides the **fitness function** for the Evolutionary Cycle (Phylogenetics).

Fitness Landscape: In evolutionary biology, organisms traverse a fitness landscape where peaks represent high survival rates. In WAF T, this landscape is strictly defined by the D&D DC Scale.

An agent’s “fitness” is its aggregate success rate across DCs 5, 10, 15, 20, and 25.

Drift vs. Selection:

Type	Description
Selection	Positive selection occurs when an agent solves a DC 25 task that its parent failed. This fixes the “mutation” (the new prompt/code) in the lineage.
Drift	If an agent solves a DC 10 task that its parent also solved, but with a different code structure (synonymous mutation), this is Genetic Drift .

Table 8: Selection vs. Drift in WAFT

Phylogenetic Signal: By analyzing the tree produced by Genotypst, researchers can detect which “Ability Scores” (from the D&D model) are **phylogenetically conserved**.

For example, does high “Wisdom” (Safety) tend to be inherited, or is it easily lost during optimization for “Intelligence” (Speed)?

This uses metrics like:

- **Pagel’s Lambda** (λ)
- **Blomberg’s K**

to measure the “heritability” of prompt strategies.

Interaction: Literate Programming $\leftrightarrow$ Typst

This interaction forms the **User Interface** and **Storage Layer** of WAFT.

The Problem

Evolving agents often become “black boxes” of spaghetti code and incomprehensible prompt chains. As the system evolves, the **rationale** for specific prompts is lost.

The Solution

By enforcing Literate Programming, every mutation must be accompanied by a **narrative explanation** generated by the LLM.

The agent cannot just change a prompt; it must write a paragraph explaining **why**.

Typst as the Enforcer: The Typst compiler can be set up such that the code will not compile if the documentation coverage drops below a certain percentage (measured via introspection).

Living Visualization: The fletcher package allows the document to contain **dynamic diagrams** of the agent’s logic. If the agent changes its decision tree (via Reflexion), the Fletcher code in the literate source is updated, and the resulting PDF shows the new flowchart automatically.

This creates a **“Self-Updating Blueprint”** that matches the physical code.

Interaction: Quines $\leftrightarrow$ The Evolutionary Cycle

The concept of the Quine is the “**engine**” of the cycle.

Traditional AI	The model output is distinct from the model definition.
WAFT AI	The model output is the model definition for the next generation.

Table 9: Traditional AI vs. WAFT AI

Mechanism:

1. The agent script imports `sys` and `inspect`
2. It reads `__file__` (its own source code)
3. It performs the task (Scint Gym)
4. It appends the results (Gym Scores, Phylogenetic Tree Newick String) to the metadata of the source code variable
5. It writes to `sys.stdout` $\rightarrow$ `agent_gen_N+1.py` (or `.typ`)

This ensures that the **Ancestral State** is physically embedded in the child, preserving the phylogenetic chain without an external database.

Core Insight

The code itself is the database.

End of Theoretical Foundations

“The architecture described herein represents a convergence of biological evolution, formal logic, game theory, and computational self-awareness into a unified framework for autonomous cognitive agents.”